AAGAM Web (Next.js) Role Simulation Guide

Running the single Next.js web app and walking every role surface — Admin, Store, Rider and Customer — through the browser, end to end.

Repository	Saikumar-bali/AAGAM_E-commerce
App	`apps/admin-dashboard` (Next.js 15 App Router) — serves Shop, Store, Rider and Admin surfaces from one build
Gateway	`apps/api-gateway` (NestJS) on `:3005`, proxied at `/api/*`
Database	Supabase Postgres — `DATABASE_URL=$AAGAAM_DATABASE`
Method	Headless Chromium (Playwright) driving the real UI — no direct API calls for the walkthrough
Date	2026-10-11

1. Role logins (all four surfaces)

One login page (`/login` , password mode) routes each account to its own portal by role. These are the accounts seeded into the sandbox database and used for this simulation. The password shown is the sandbox demo password set for this guide.

Role	Login (email)	Password	Lands on	Surfaces exercised
ADMIN	`admin@gmail.com`	`Demo@2026#`	`/admin`	Dashboard, Riders, Orders, Subscriptions
STORE	`store@aagam.com`	`Demo@2026#`	`/store`	Dashboard, Milk Grid, Deliveries, Orders
RIDER	rider account (role `RIDER`)	`Demo@2026#`	`/rider`	Dashboard, Runs, Run Console, History, Pickup
CUSTOMER	`customer1@gmail.com`	`Demo@2026#`	`/shop`	Shop, Addresses, Subscriptions, Orders

> **Credentials are environment-provided.** These accounts and passwords exist only in the local sandbox database; nothing here is a production credential. Store the real logins in `AAGAM_*` environment variables per the `aagam-testing` skill — never in the repository.

2. Running the stack locally

```
# 1. shared packages (types, utils, database) — the dashboard imports them
npm run build --workspace=packages/types
npm run build --workspace=packages/utils
npm run build --workspace=packages/database   # also runs prisma generate

# 2. api-gateway on :3005 against the sandbox database
cd apps/api-gateway
export DATABASE_URL="$AAGAAM_DATABASE" JWT_SECRET=<32+ chars> REDIS_URL=redis://127.0.0.1:6379
# PLAYWRIGHT_QA=true lifts the /auth/login throttle (3/min) so scripted logins don't 429
PLAYWRIGHT_QA=true node dist/src/main.js

# 3. dashboard: /api/* must be proxied to the gateway, and the Mapbox token baked in at build
cd apps/admin-dashboard
cat > .env.local <<'EOF'
NEXT_PUBLIC_API_URL=/api
API_BACKEND_URL=http://127.0.0.1:3005
NEXT_PUBLIC_MAPBOX_TOKEN=pk.<your public token>
EOF
npx next build
NEXT_PUBLIC_API_URL=/api API_BACKEND_URL=http://127.0.0.1:3005 npx next start -p 12000
```

Two build-time gotchas. `NEXT_PUBLIC_API_URL` is baked into the client bundle at *build* time — it must be the same-origin path `/api` so the browser calls the dashboard's own proxy, not `localhost:3005`. And `NEXT_PUBLIC_MAPBOX_TOKEN` must be present during `next build`; otherwise the rider run console and live-tracking maps render the "Map unavailable – missing Mapbox token" placeholder.

3. How the four surfaces fit together

The single Next.js build serves every browser role by route group; the gateway is the only backend.

Surface	Route group	Entry route	Gateway prefix
Shop (customer)	`(shop)/shop`	`/shop`	`/shop/*`, `/customer/*`
Store portal	`(store)/store`	`/store`	`/store/*`
Rider portal	`(rider)/rider`	`/rider`	`/rider/*`, `/riders/*`
Admin dashboard	`(admin)/admin`	`/admin`	`/admin/*`
Auth	`(auth)`	`/login`	`/auth/*`

4. Login — the shared entry point

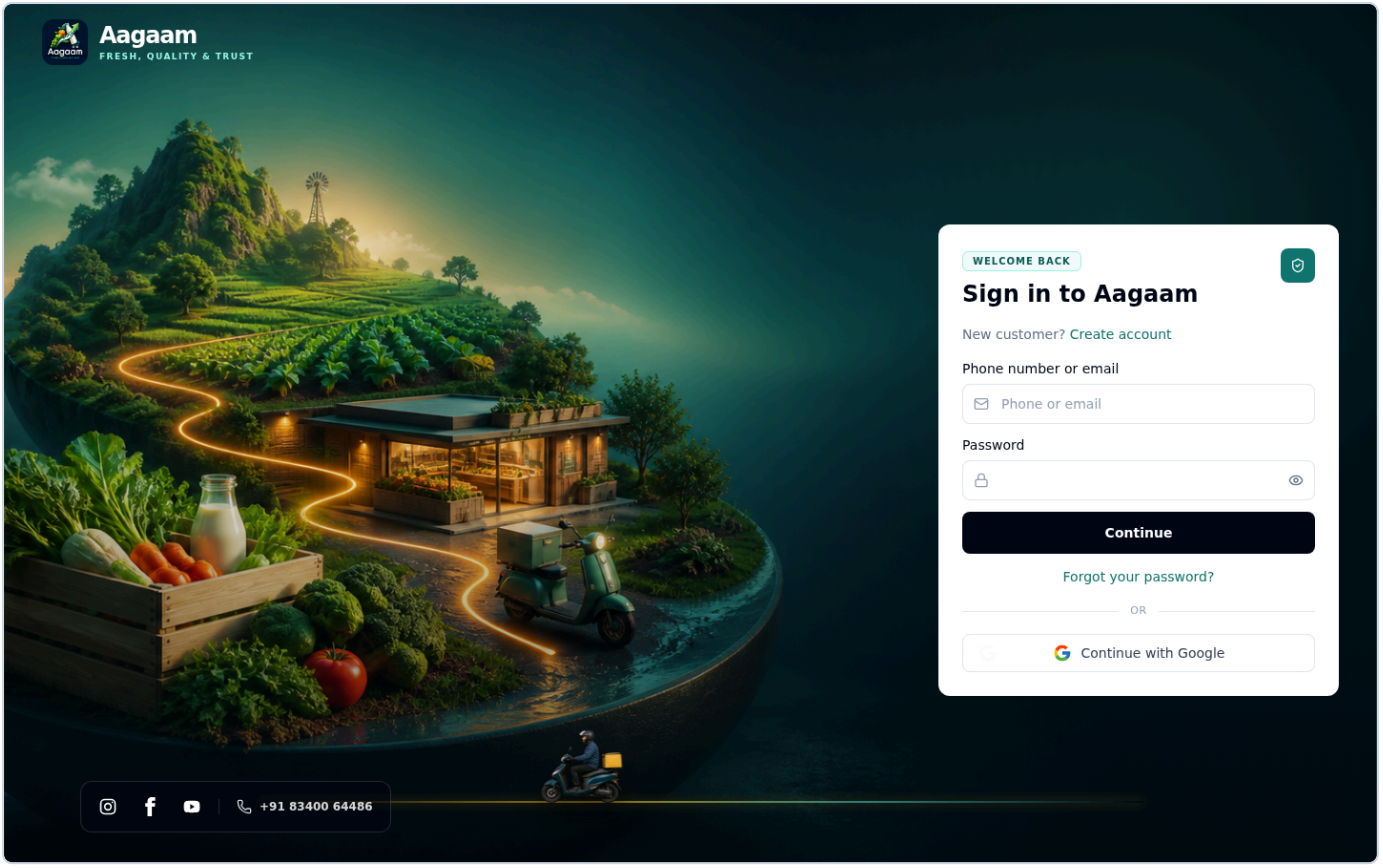


Figure 1 — `/login` in password mode. Submitting an email + password routes to `/admin`, `/store`, `/rider` or `/shop` by the account's role.

5. ADMIN surface

Login as `admin@gmail.com` → lands on `/admin`. The admin portal is the operations control plane: one store, one rider, live order and subscription counts.

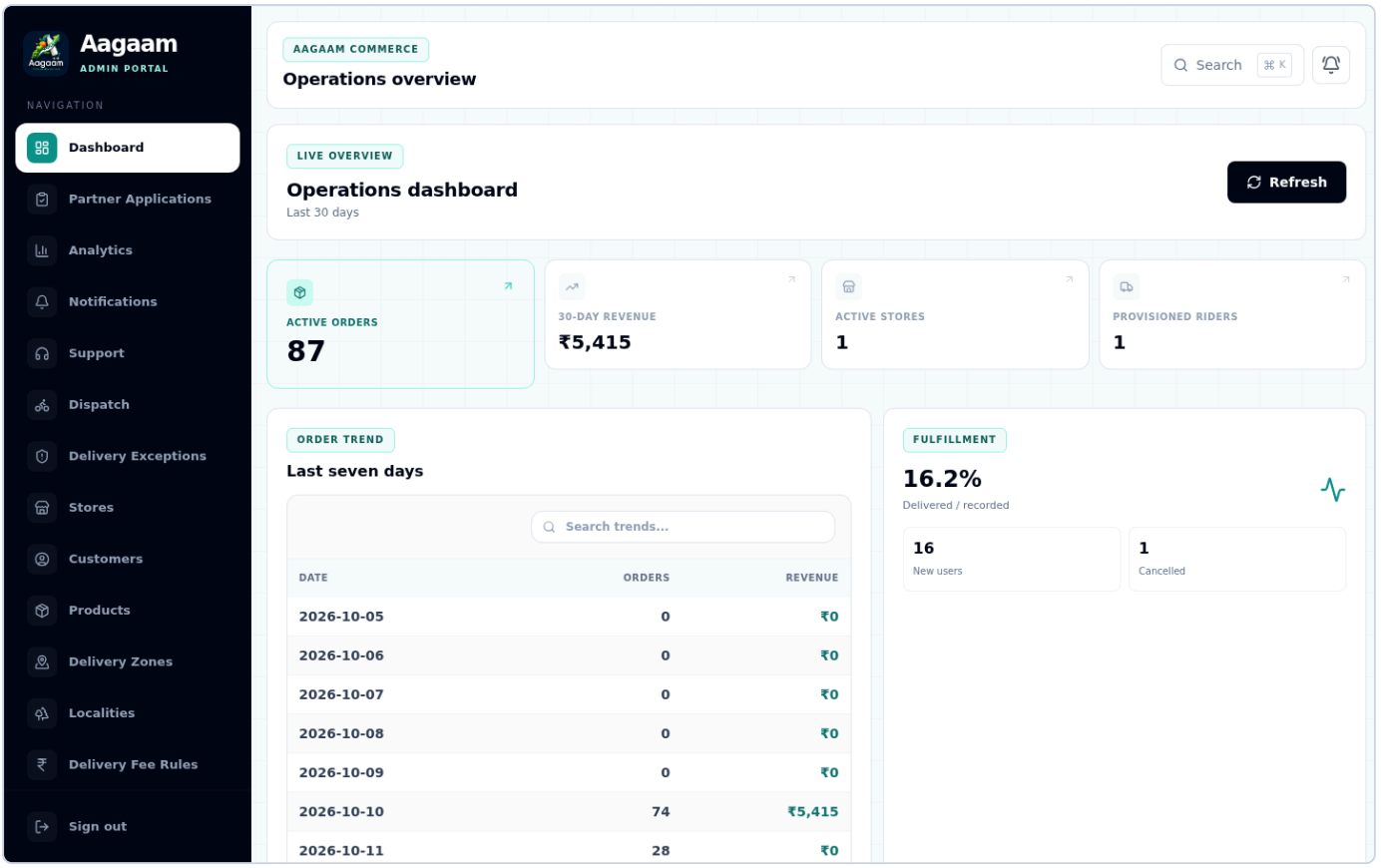


Figure 2 — `/admin`: operations overview (active orders, 30-day revenue, active stores, provisioned riders).

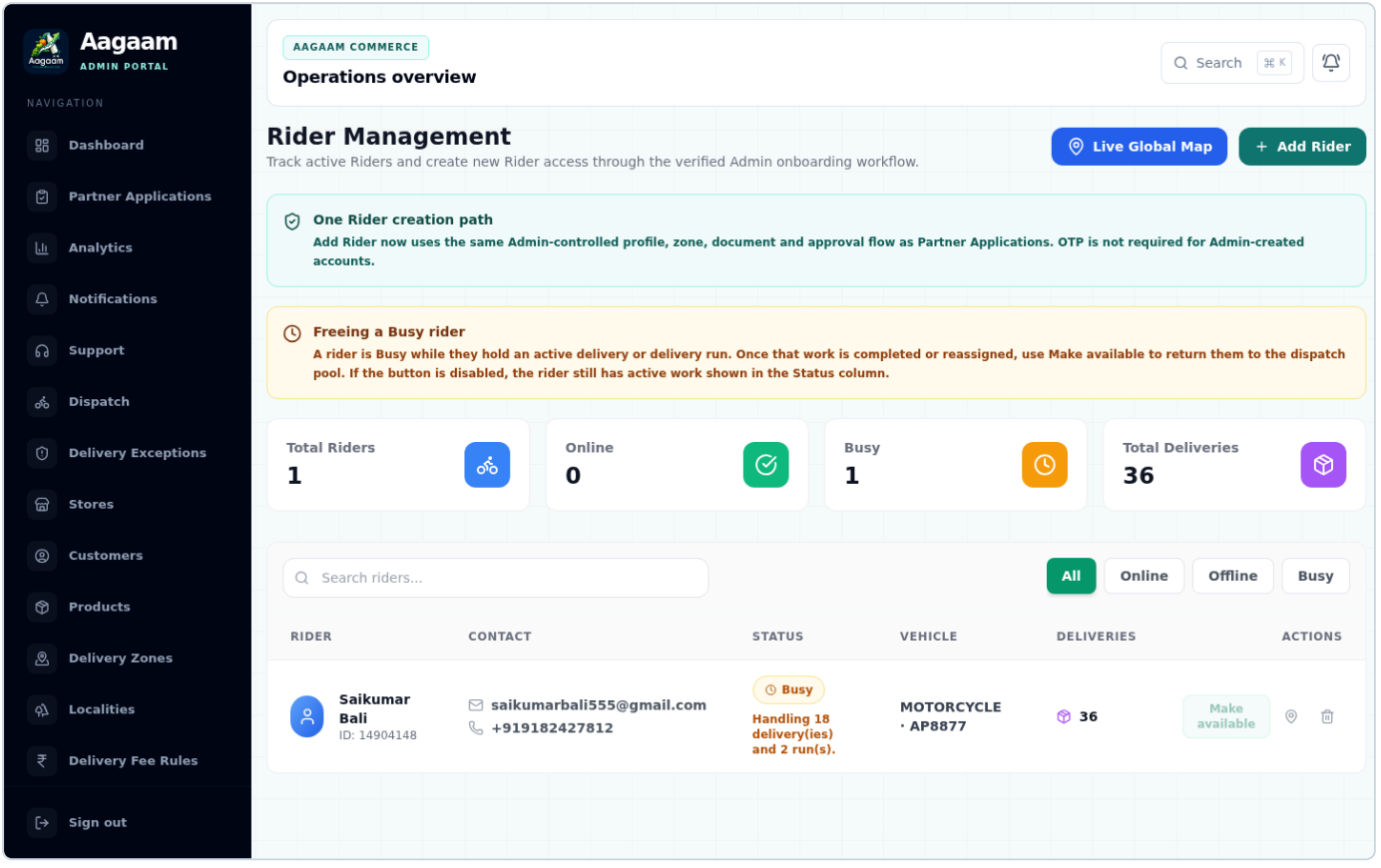


Figure 3 — `/admin/riders`: rider management, workload and the Busy/Online release controls.

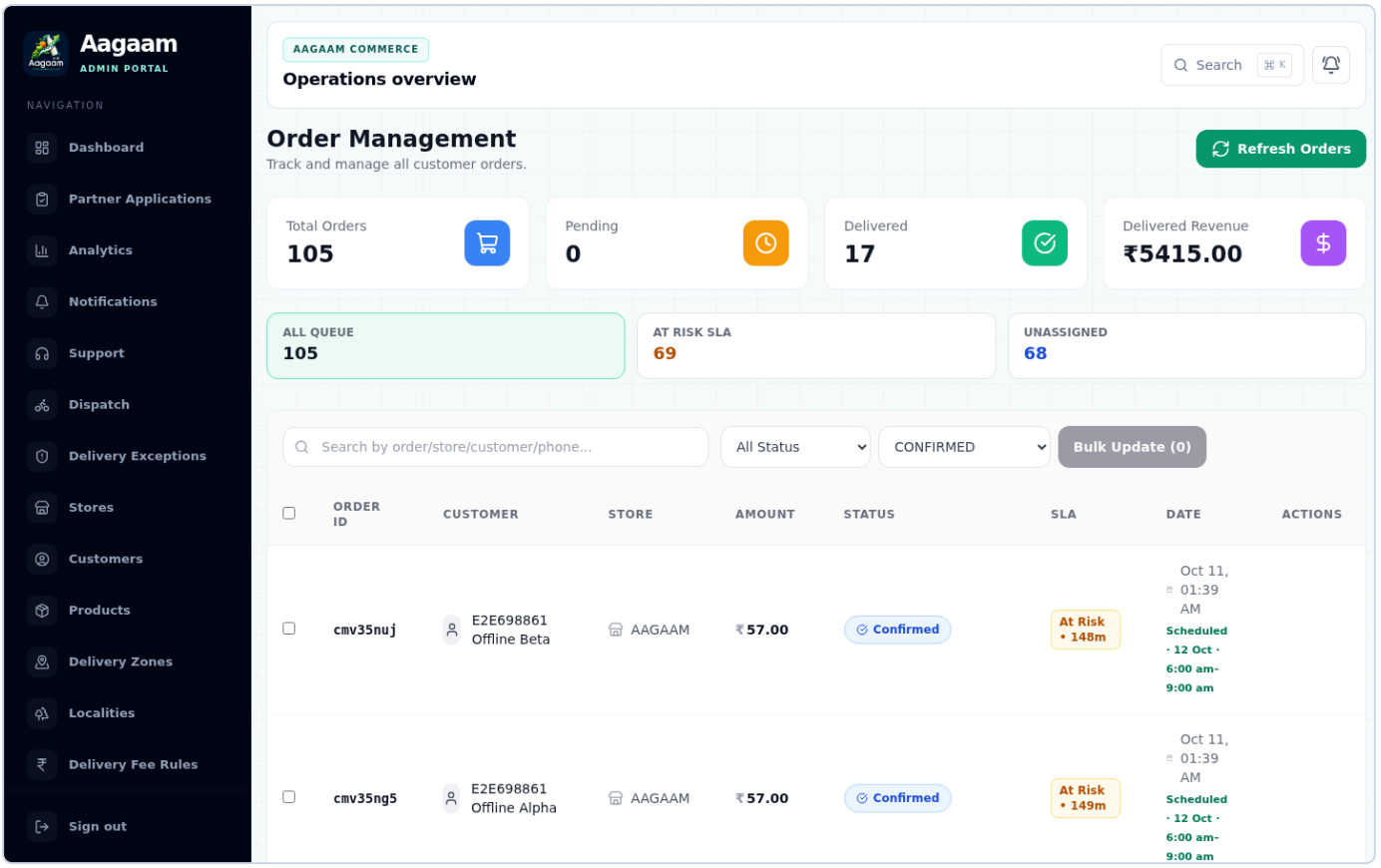


Figure 4 — `/admin/orders`: order management with queue / delivered tabs and revenue.

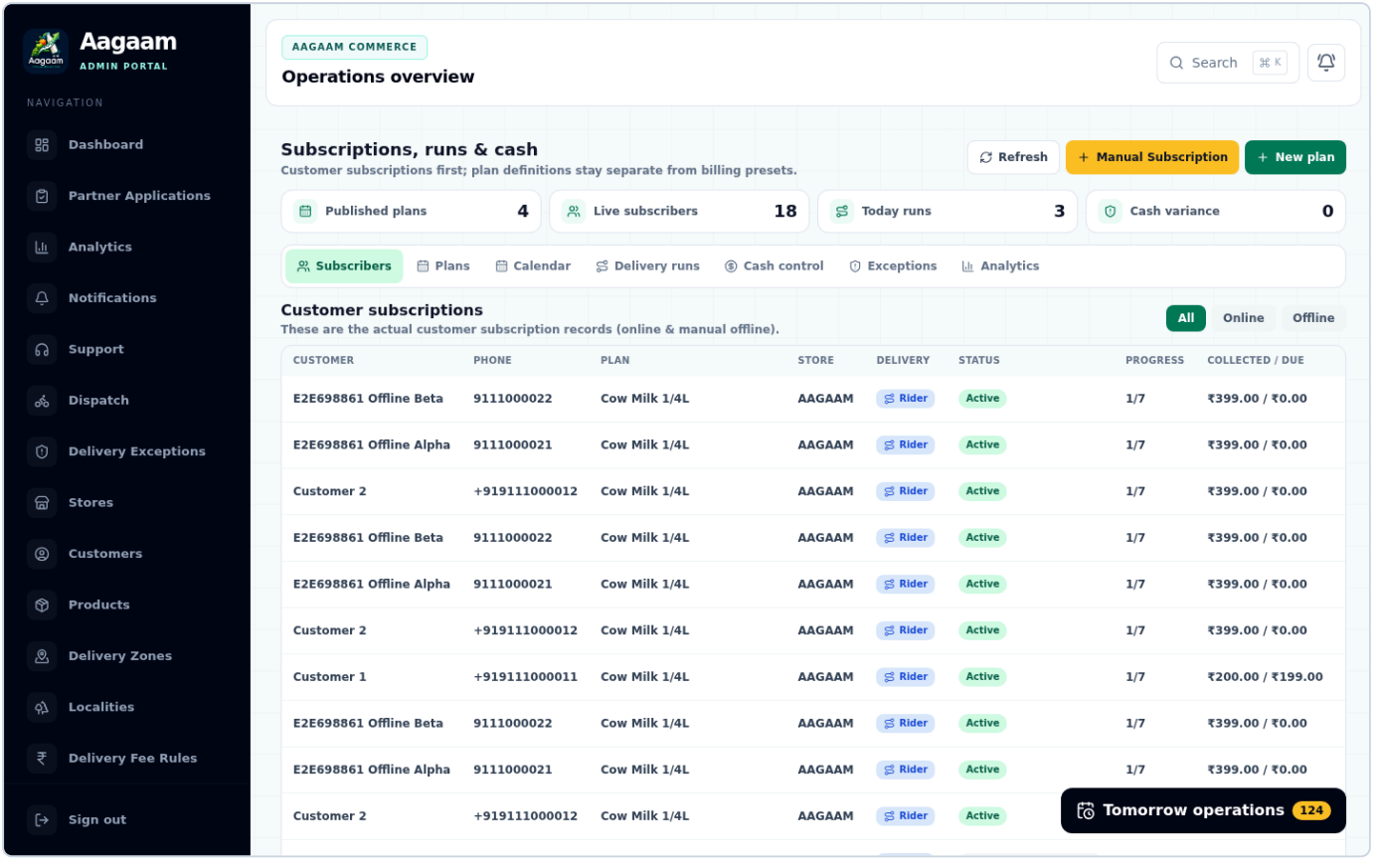


Figure 5 — `/admin/subscriptions`: customer subscriptions, runs and cash.

6. STORE surface

Login as `store@aagam.com` → lands on `/store`. The store workspace owns the milk board, subscription hub and delivery runs.

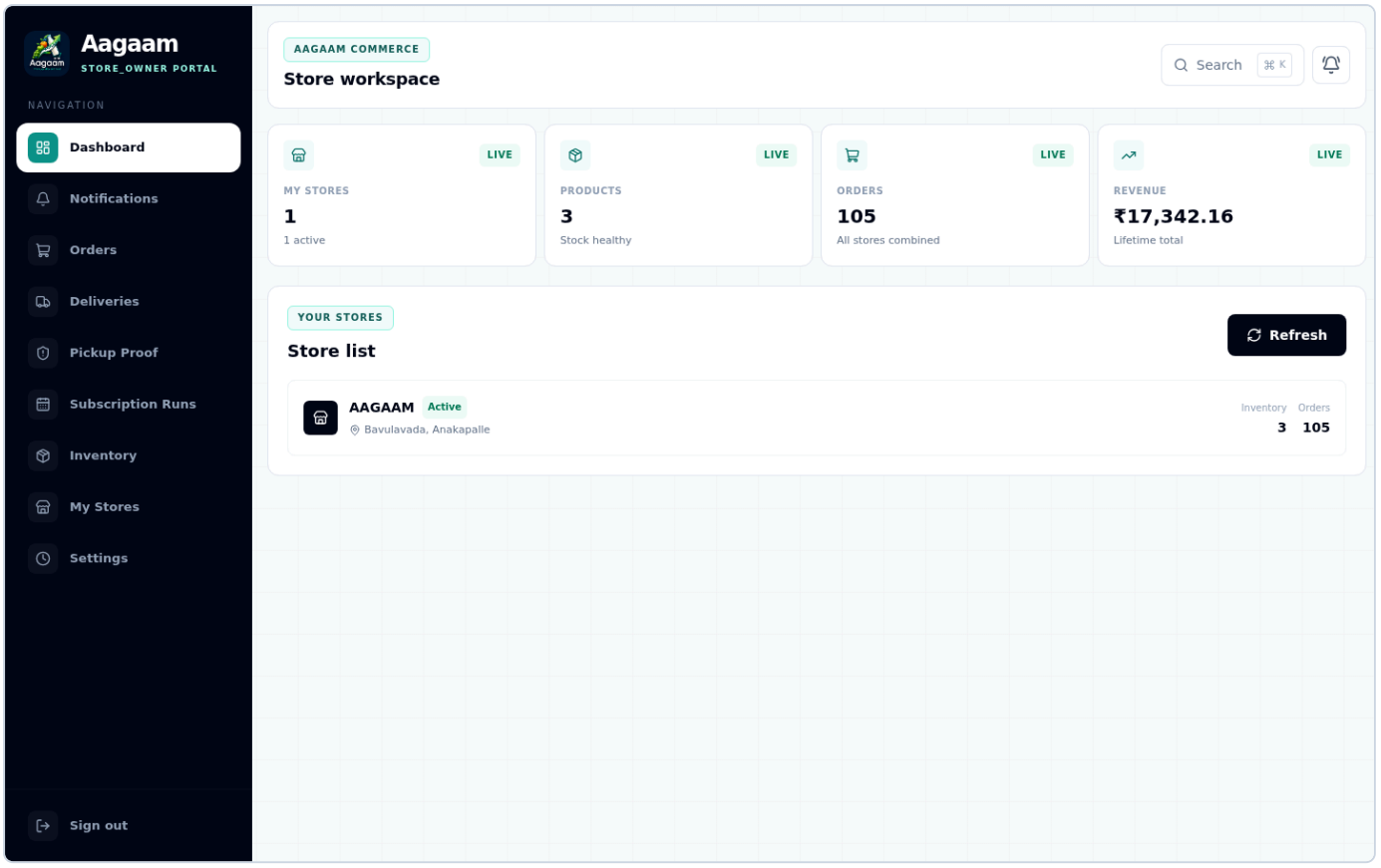


Figure 6 — `/store` : store workspace (stores, products, live orders, lifetime revenue).

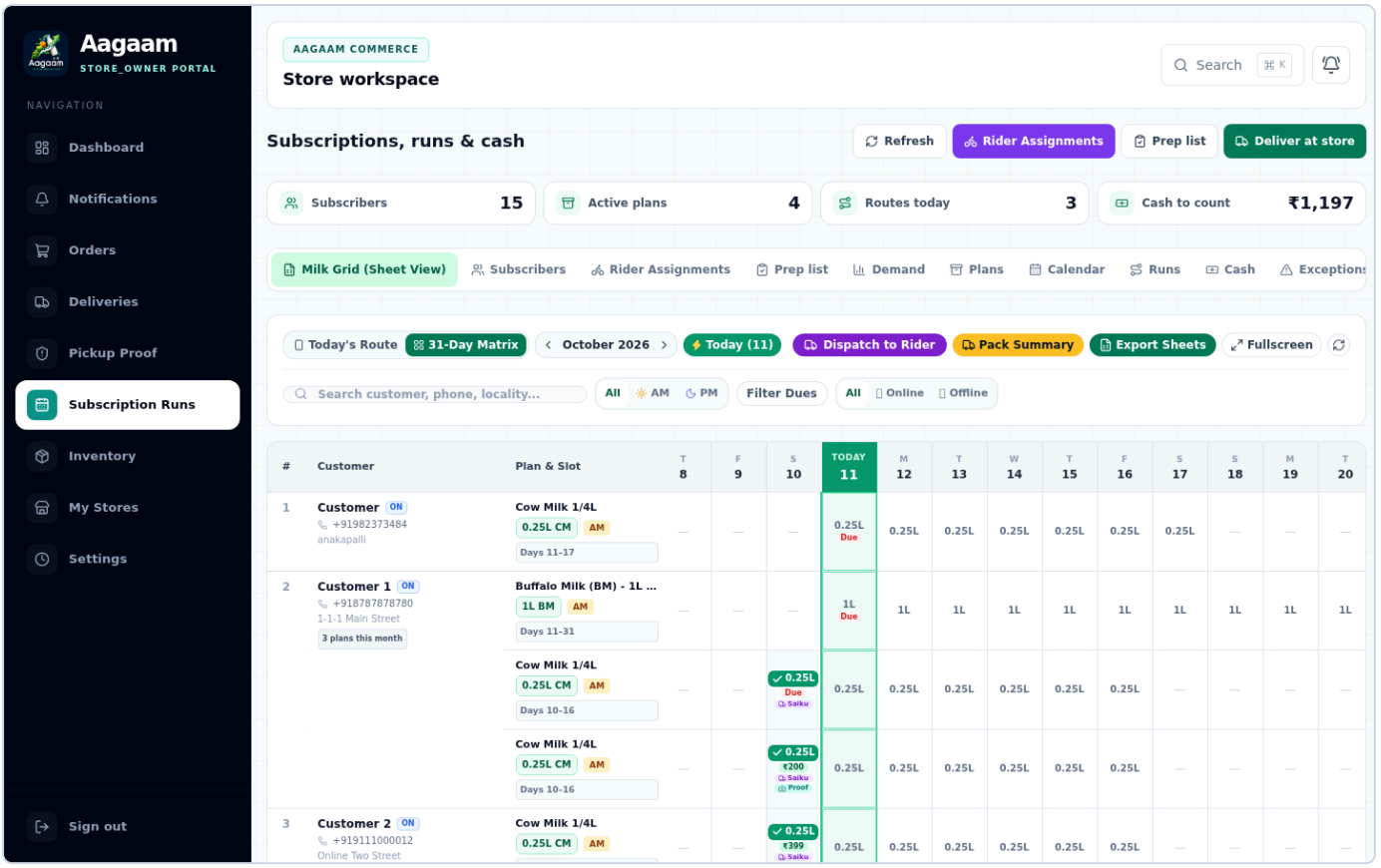


Figure 7 — `/store/subscriptions` : the subscription hub. Tabs for Milk Grid, Rider Assignments, Prep list, Demand, Plans, Calendar, Runs, Cash; each customer appears once with plans stacked.

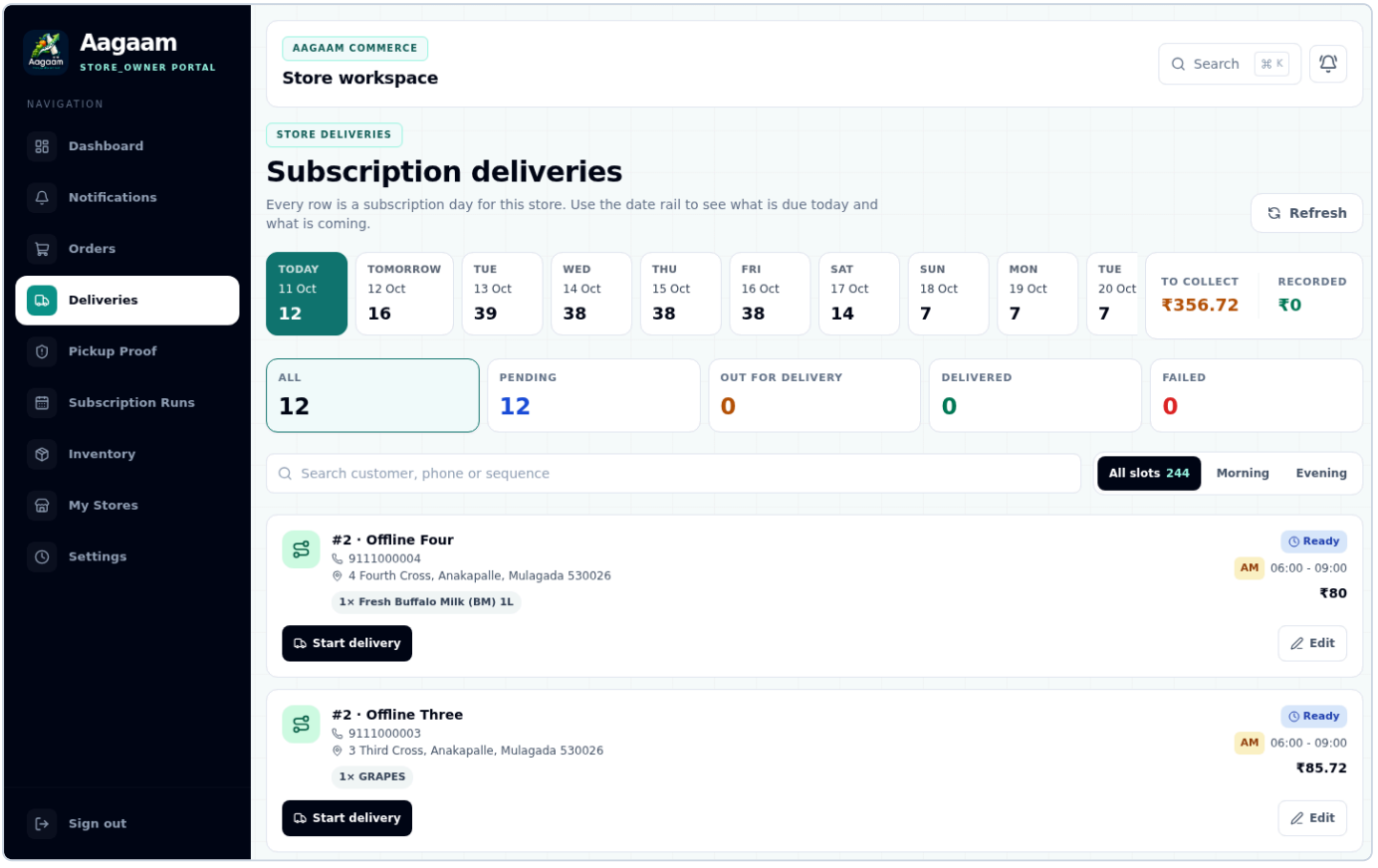


Figure 8 — `/store/deliveries` : every subscription day for the store, with a date rail (today / tomorrow / week).

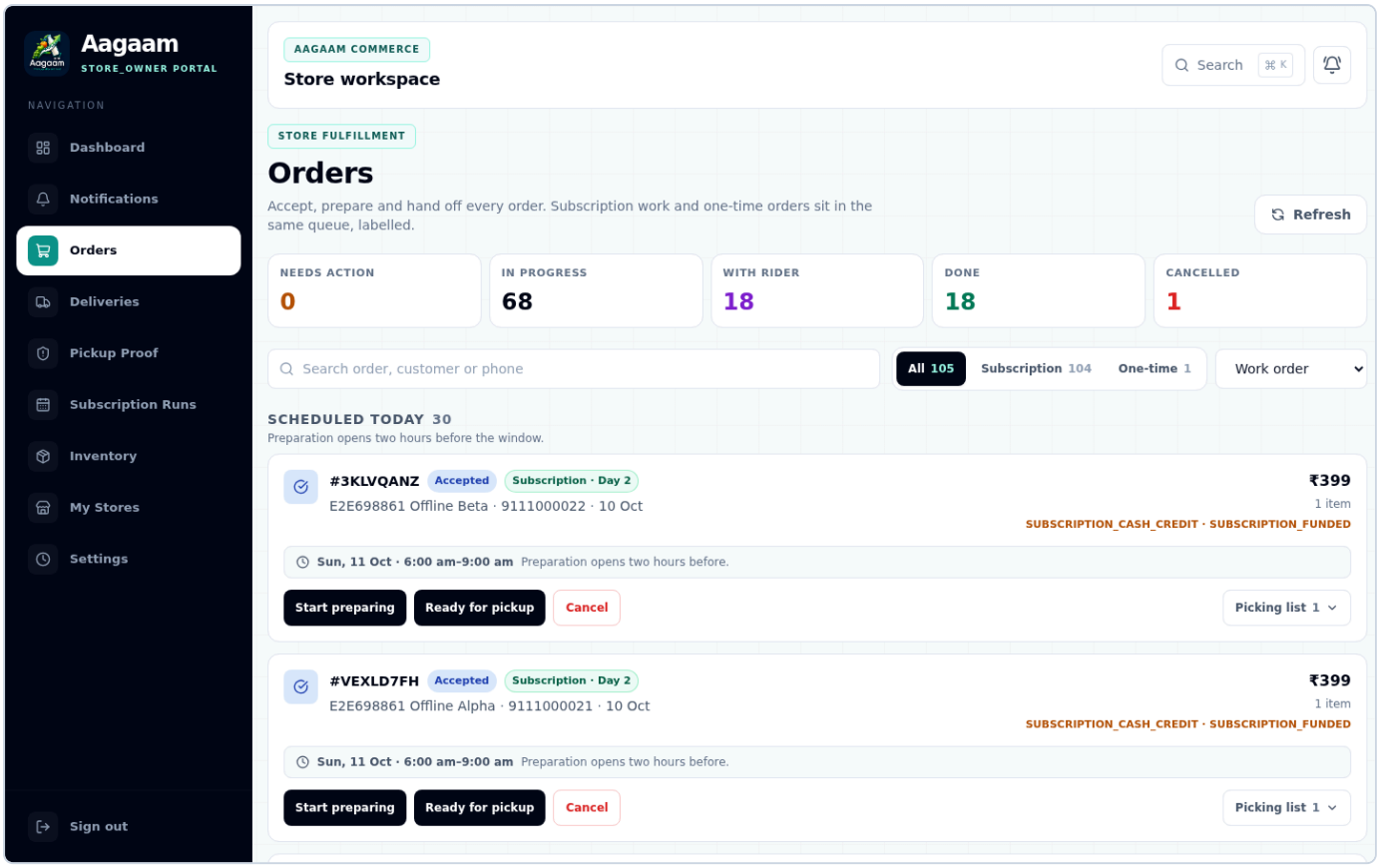


Figure 9 — `/store/orders`: fulfilment queue — subscription and one-time orders labelled in the same board.

7. RIDER surface (delivery flow)

Login as the rider account → lands on `/rider`. This is the delivery workspace: offers, the active delivery, subscription delivery runs and the stop-by-stop run console.

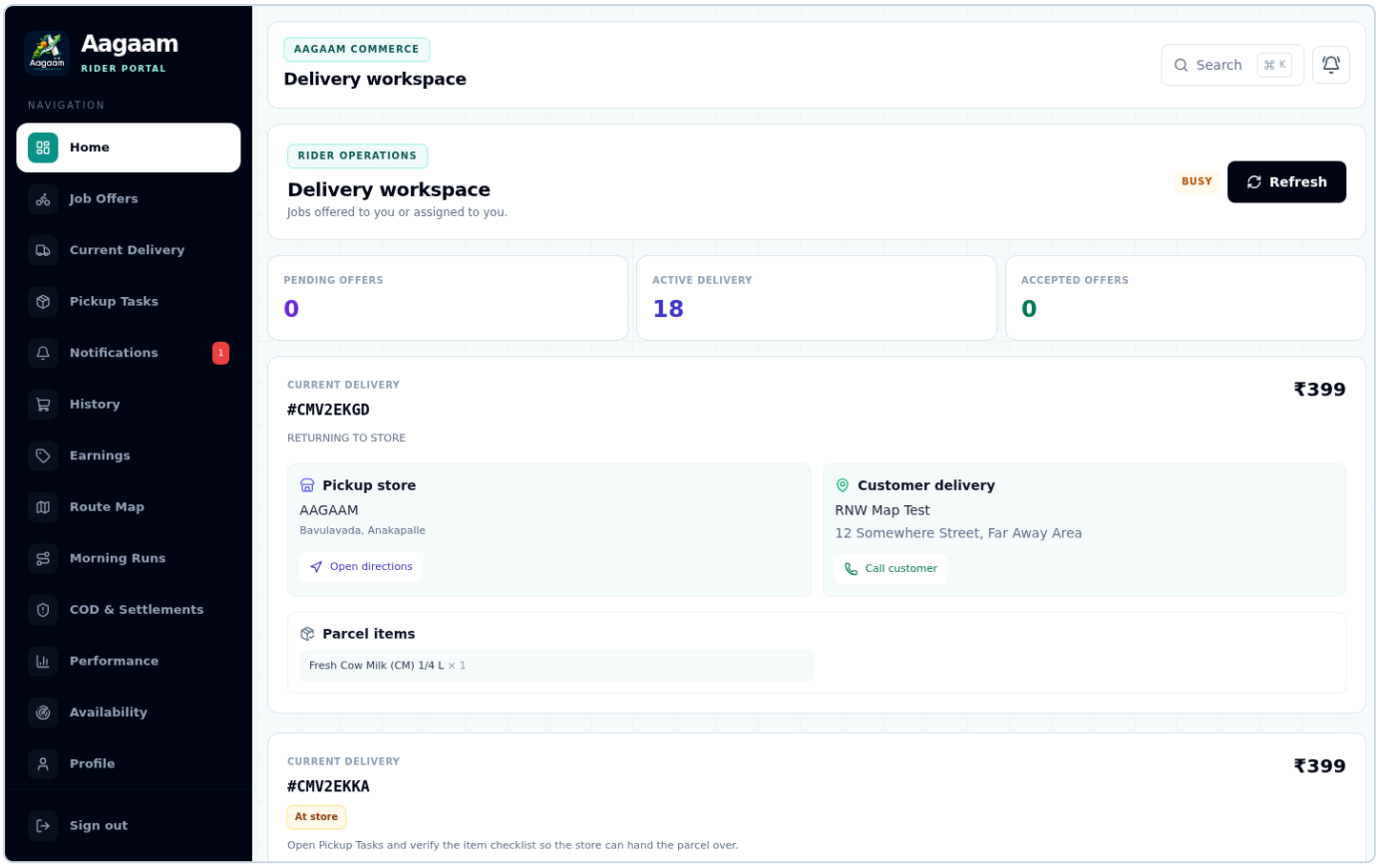


Figure 10 — `/rider`: delivery workspace — pending offers, active delivery and the current delivery card.

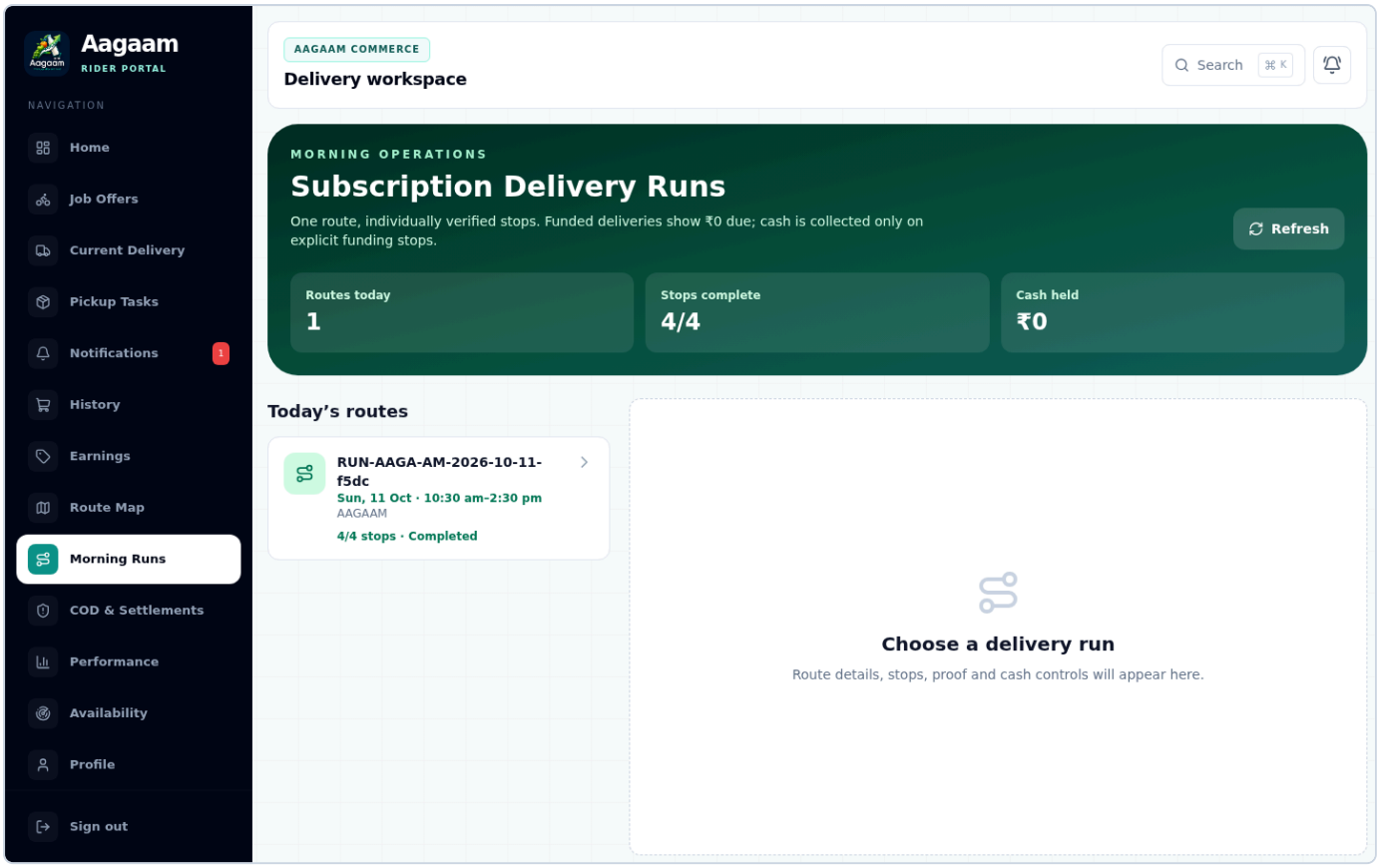


Figure 11 — `/rider/runs` : today's subscription delivery run — one route, individually verified stops, cash held.

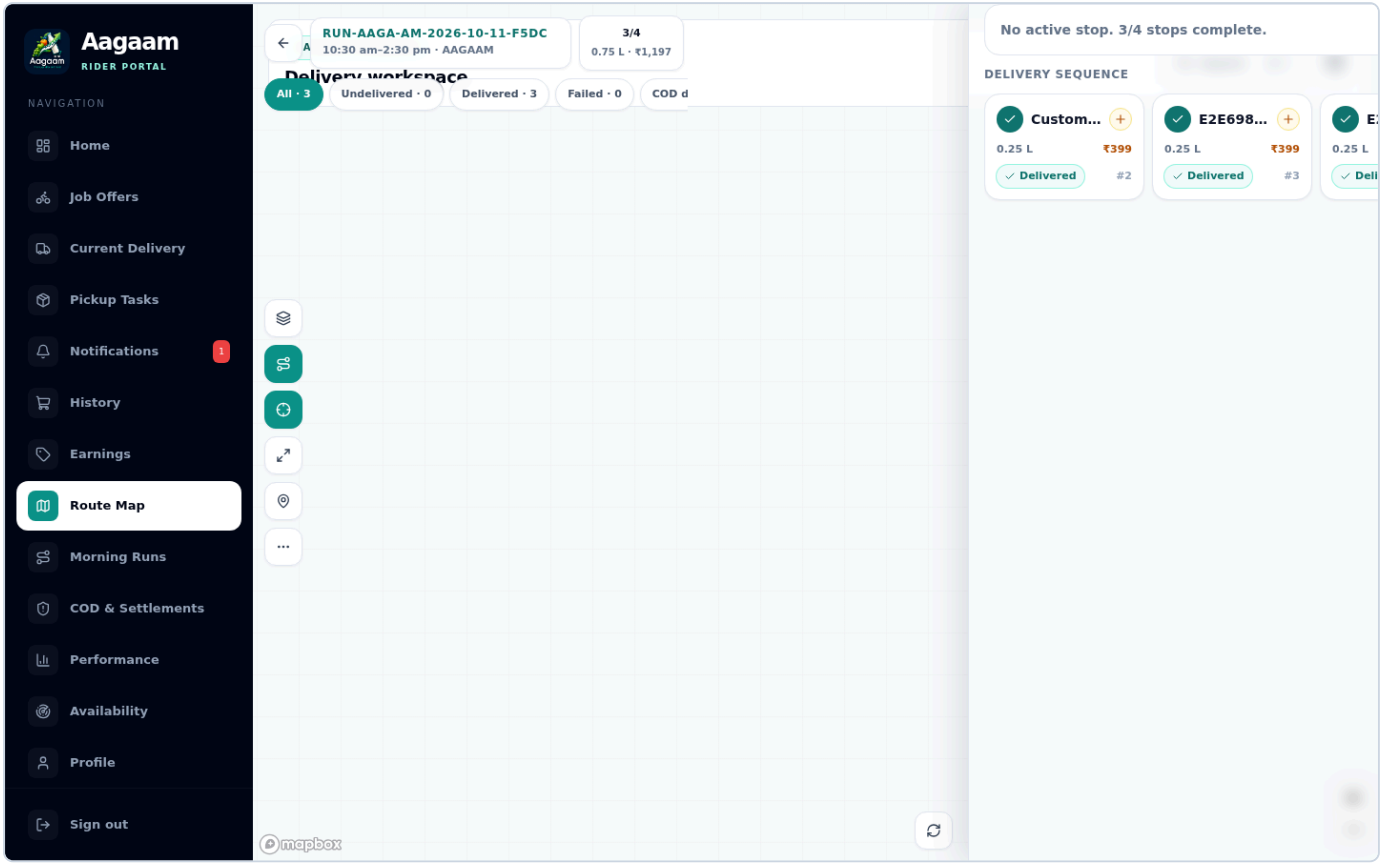


Figure 12 — `/rider/console` : the run console — route progress (3/4 done), per-stop cards (Customer 2 · 0.25 L · ₹399 · Delivered) and the delivery sequence. The map is a live Mapbox canvas when a valid token is baked in.

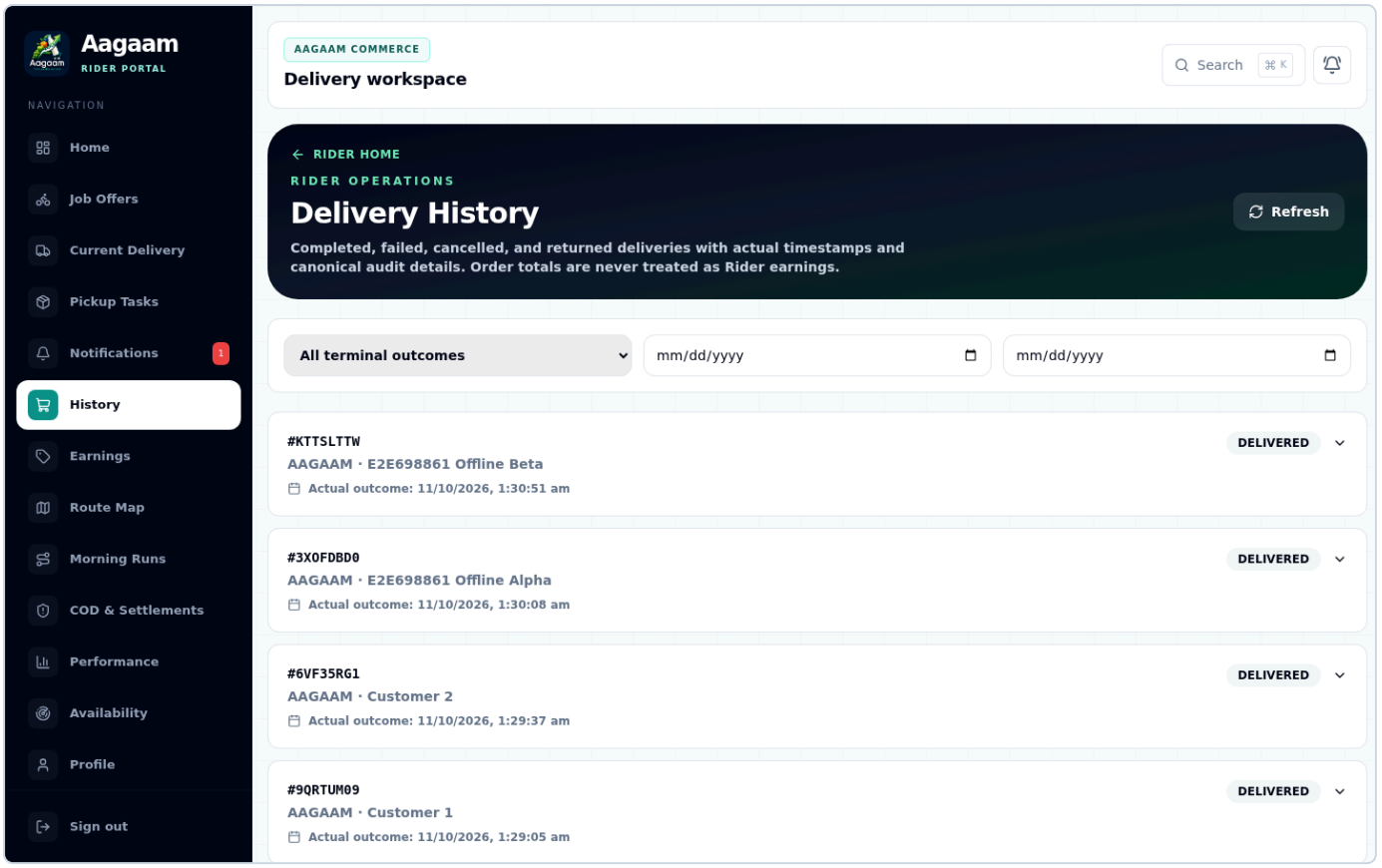


Figure 13 — `/rider/history`: terminal outcomes (completed / failed / cancelled / returned) with timestamps and audit detail.

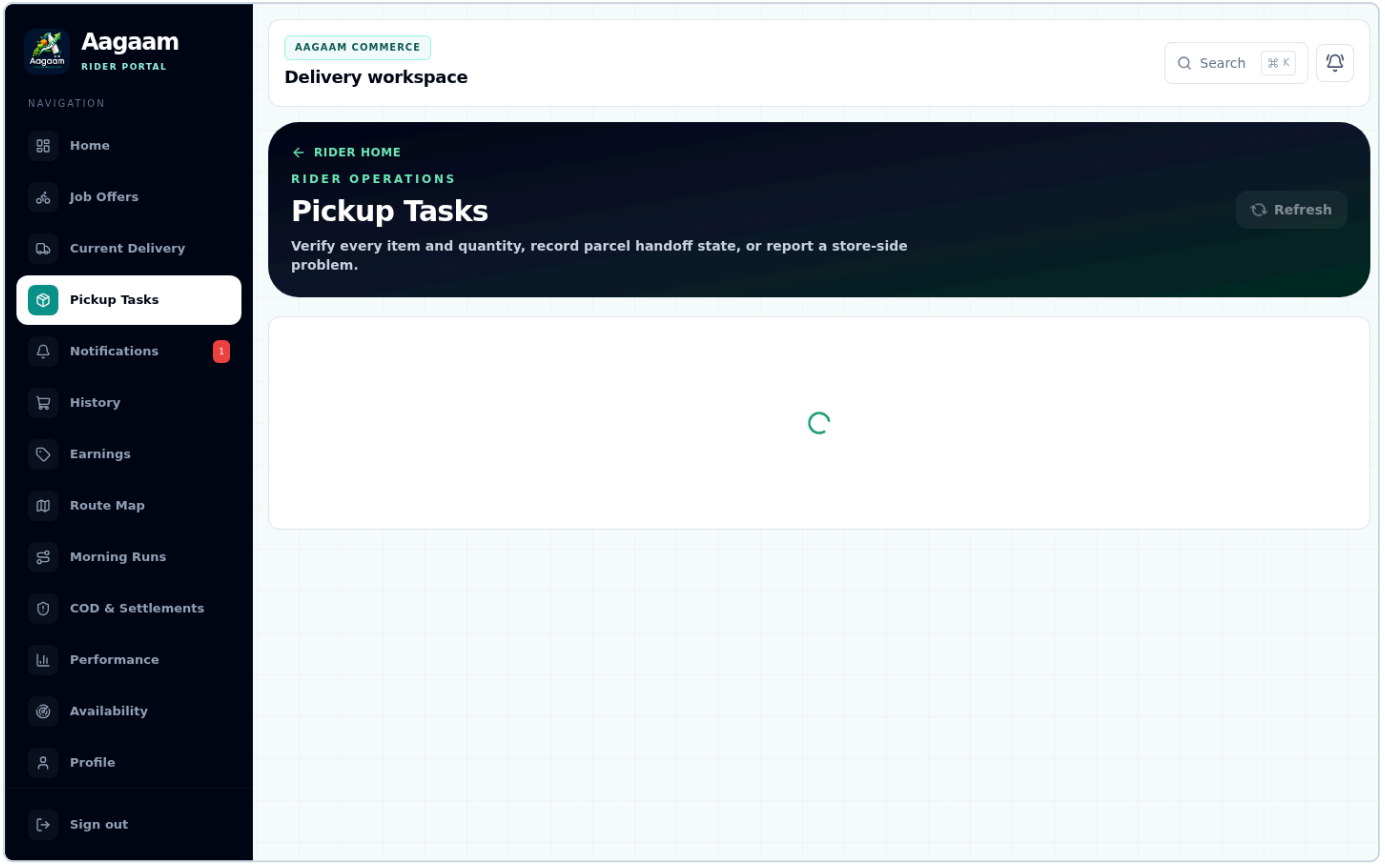


Figure 14 — `/rider/pickup`: pickup tasks — verify items and quantities, record parcel handoff or report a store-side problem.

Rider delivery flow, end to end. Pickup (`/rider/pickup` : verify bag → *Start run*) → route (`/rider/runs` : open the run) → stop console (`/rider/console` : *Arrive* records GPS, then proof — OTP for security-reception, or photo+GPS for personal handover/trusted drop — then *Complete*, optionally recording COD cash) → accountability (cash batch) →

history (`/rider/history` shows the terminal outcome). Each completion advances the linked order via the shared delivery workflow, not a raw stop update.

8. CUSTOMER surface

Login as `customer1@gmail.com` → lands on `/shop`. The shop is the customer's own view of the same orders and subscriptions the store and rider act on.

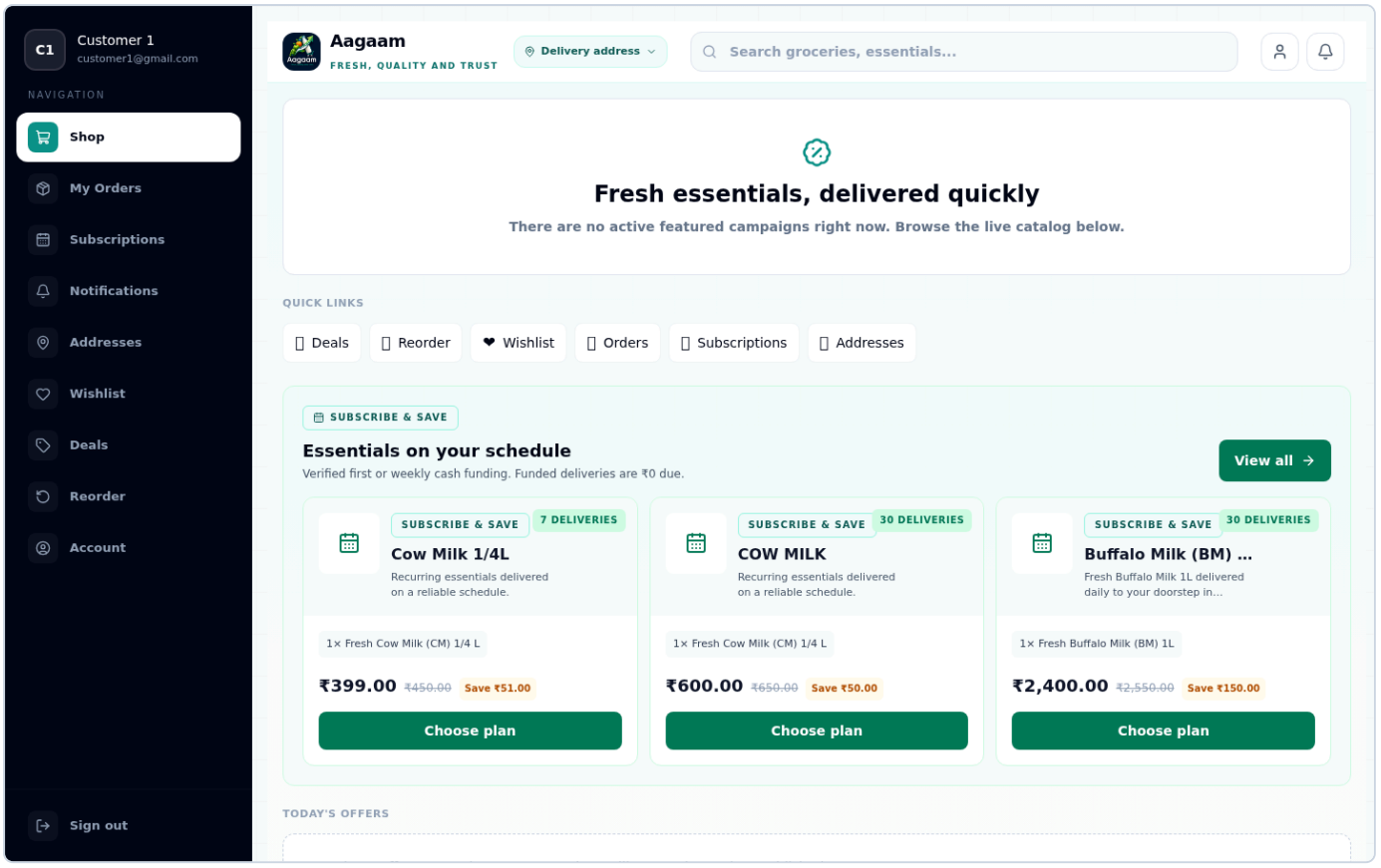


Figure 15 — `/shop` : the customer shop with subscribe-and-save entry points.

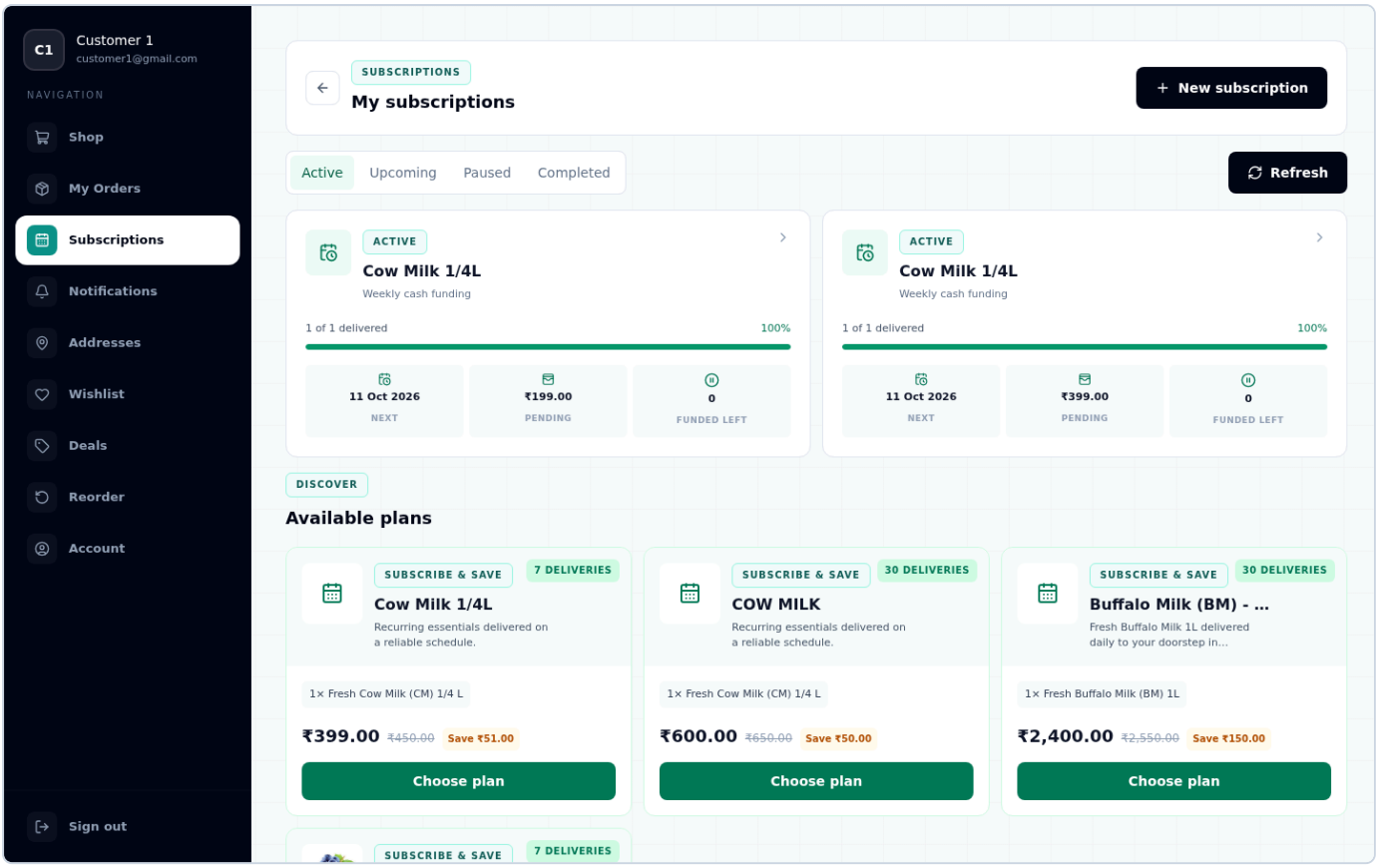


Figure 16 — `/shop/subscriptions`: the customer's active subscriptions (plan, funding mode, deliveries delivered, next amount) and the plan catalog.

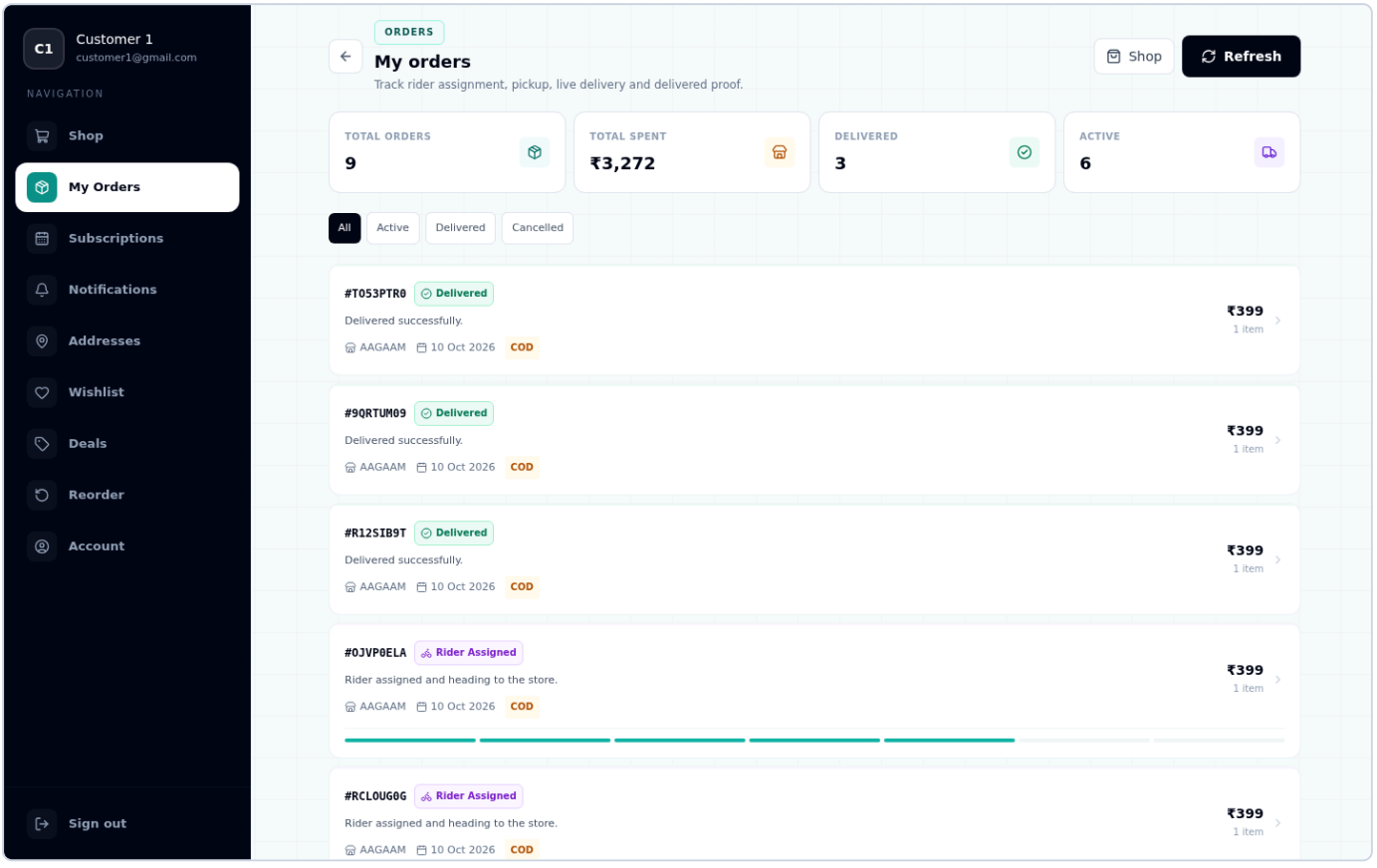


Figure 17 — `/shop/orders`: the customer's orders with rider assignment, pickup, live delivery and delivered proof.

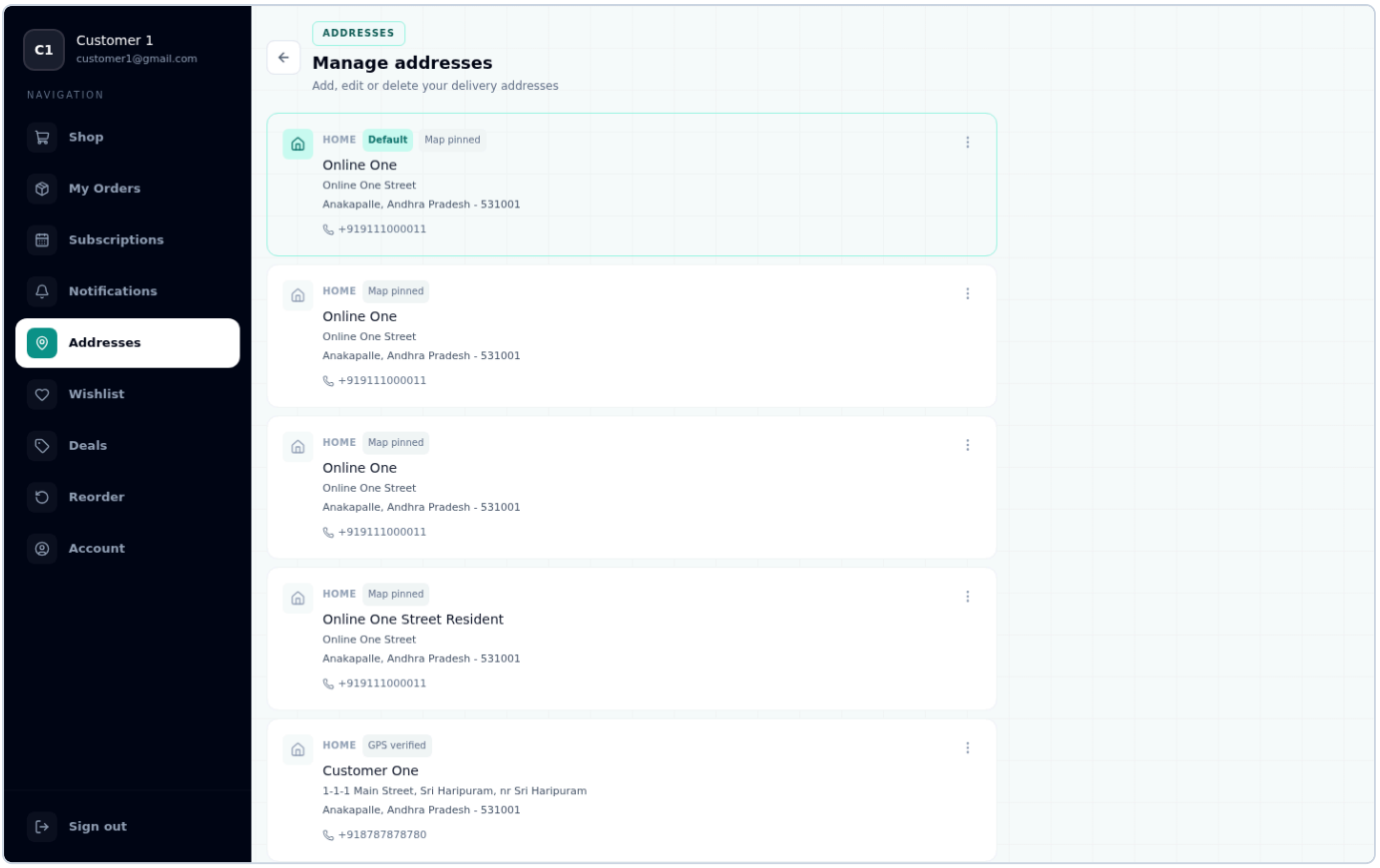


Figure 18 — `/shop/addresses`: saved delivery addresses used by subscriptions and orders.

9. Cross-role simulation (one delivery, four views)

The value of walking all four roles is seeing the same object from each side. This sequence was driven entirely through the browser.

#	Role	Action in the UI	Observed
1	CUSTOMER	Open `/shop/subscriptions`; read an active subscription	Plan, funding mode, *N of M delivered*, next amount
2	STORE	Open `/store/subscriptions` → Milk Grid; find the same customer	One row per customer, plans stacked, per-day litres and Paid/Due
3	RIDER	Open `/rider/runs` → the run → `/rider/console`	Same deliveries as verified stops on the route, with cash due
4	RIDER	Arrive → proof → Complete (record COD if any)	Stop advances; run progress increments
5	CUSTOMER	Open `/shop/orders`	Order shows the updated status / delivered proof
6	STORE	Open `/store/deliveries` for the date	The day's cell reflects the completed delivery
7	ADMIN	Open `/admin/subscriptions` and `/admin/orders`	Reconciled subscription balance and order status

10. Verification checklist

Check	How (UI)	Result
App serves	Load `/login`, `/shop`, `/store`, `/rider`, `/admin`	200
Admin login	`admin@gmail.com` → `/admin`	lands
Store login	`store@aagam.com` → `/store`	lands

Rider login	rider account → `/rider`	**lands**
Customer login	`customer1@gmail.com` → `/shop`	**lands**
Live data	Read counts/rows on each surface	**real DB data**
Rider run console	Open `/rider/console`	**route + stops render**
Console errors	Collect page + console errors during the session	**only the optional Google sign-in 403**

> **Harmless console noise.** The Google sign-in button logs a `403` / `GSI_LOGGER` entry when the page origin is not an allowed OAuth origin — it does not affect email/password login. If no valid `NEXT_PUBLIC_MAPBOX_TOKEN` was baked in, the Mapbox canvases log an invalid-token warning and show a placeholder; the rest of the run console still renders.

11. Related material

- `docs/qa/web-nextjs-guide/guide.html` — the companion UI verification guide (BUG-028 walkthrough).
- `skills/aagam-testing/` — end-to-end testing skill for the four role surfaces.
- `apps/api-gateway/src/auth/auth.service.ts` — role resolution and the master-store pass.
- `apps/admin-dashboard/src/app/(auth)/login/page.tsx` — the shared login and role-based routing.